Week 6 : Sorting (Cont.) Objectives

Upon completion of this unit, students will be able to understand and explain the following sub topics :

- Sorting
- Quick sort
- Radix Sort
- Shell sort
- Comparing algorithms

Quick Sort

- Divides an array into two pieces
  - Pieces are not necessarily halves of the array
  - Chooses one entry in the array—called the pivot
- Partitions the array

Quick Sort

- When pivot chosen, array rearranged such that:
  - Pivot is in position that it will occupy in final sorted array
  - Entries in positions before pivot are less than or equal to pivot
  - Entries in positions after pivot are greater than or equal to pivot

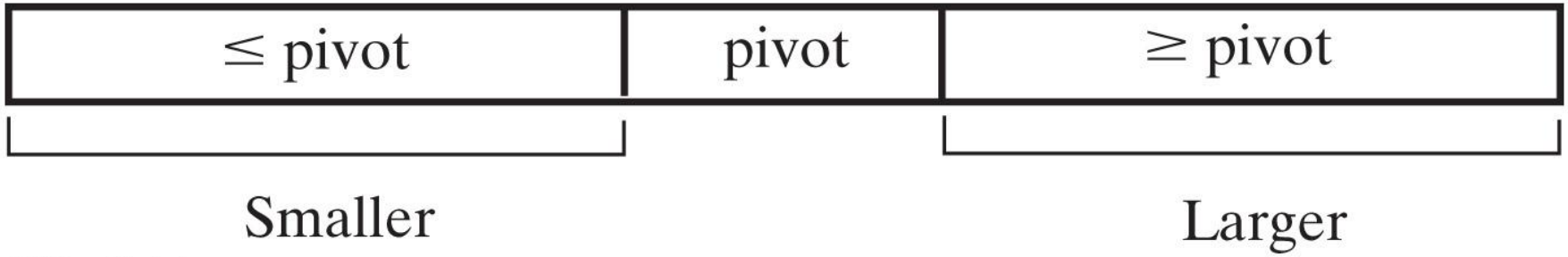
Quick Sort

```
Algorithm quickSort(a, first, last)
// Sorts the array entries a[first..last] recursively.
if (first < last)
{
        Choose a pivot
        Partition the array about the pivot
        pivotIndex = index of pivot
        quickSort(a, first, pivotIndex − 1) // Sort Smaller
        quickSort(a, pivotIndex + 1, last) //Sort Larger
}
```

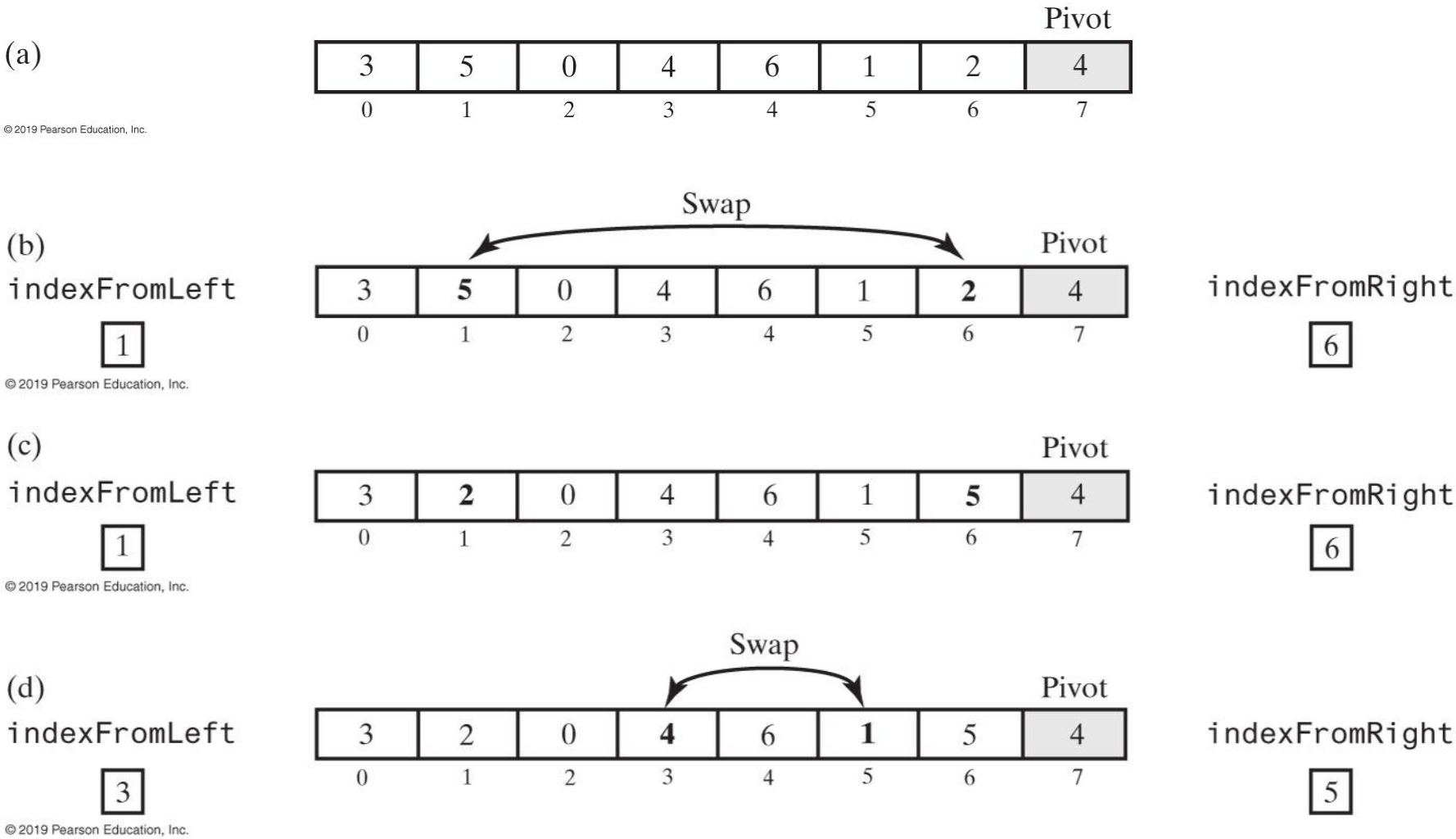
- Algorithm that describes our sorting strategy

Quick Sort

- A partition of an array during a quick sort

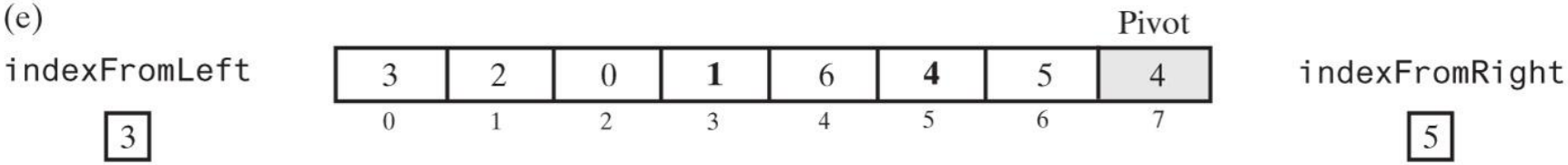


© 2019 Pearson Education, Inc.



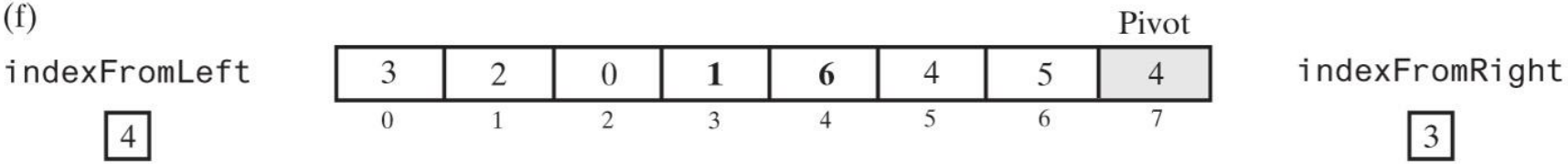
A partitioning strategy for quick sort

(e)



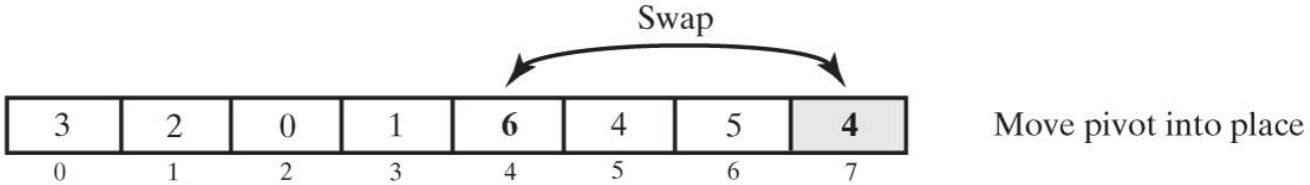
© 2019 Pearson Education, Inc.

(f)



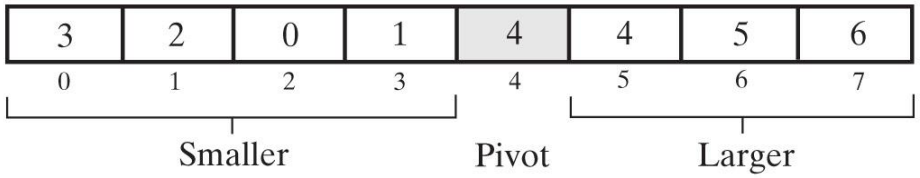
© 2019 Pearson Education, Inc.

(g)



© 2019 Pearson Education, Inc.

(h)



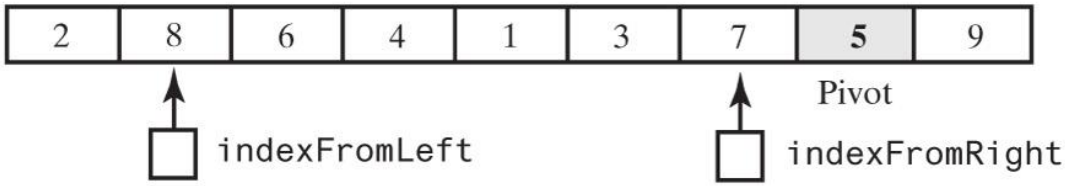
© 2019 Pearson Education, Inc.

Quick Sort Partitioning

(a) The original array

© 2019 Pearson Education, Inc.

(b) The array before partitioning and just after positioning the pivot

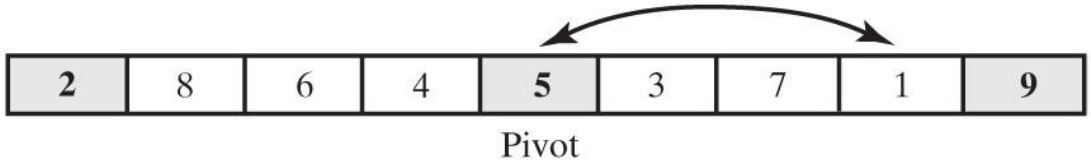


© 2019 Pearson Education, Inc.

- Median-of-three pivot selection

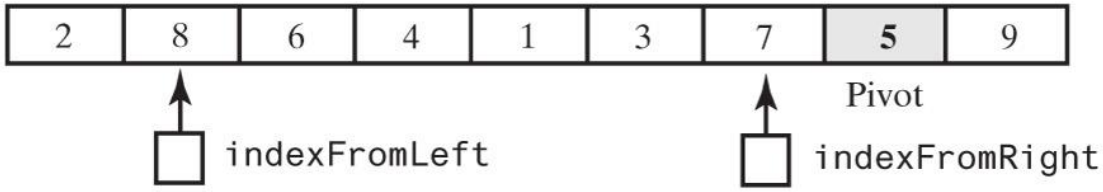
Quick Sort Partitioning

(a) The array after median-of-three pivot selection, as shown in Figure 16-7b



© 2019 Pearson Education, Inc.

(b) The array before partitioning and just after positioning the pivot



© 2019 Pearson Education, Inc.

- The array after selecting and positioning the pivot and just before partitioning

Quick Sort Partitioning (Part 1)

Algorithm **partition(a, first, last)**

// *Partitions an array* a[first..last] *as part of quick sort into two subarrays named*
// Smaller *and* Larger *that are separated by a single entry—the pivot—named* pivotValue.
// *Entries in* Smaller *are* <= pivotValue *and appear before* pivotValue *in the array*.
// *Entries in* Larger *are* >= pivotValue *and appear after* pivotValue *in the array.*
// *Precondition*: first >= 0; first < a.length; last – first >= 3; last < a.length.
// *Returns the index of the pivot.*
mid = *index of the array's middle entry*
sortFirstMiddleLast(a, first, mid, last)
// *Assertion:* a[mid] *is the pivot, that is,* pivotValue*;*
// a[first] <= pivotValue *and* a[last] >= pivotValue*, so do not compare these two*
// *array entries with* pivotValue.
// *Move* pivotValue *to next-to-last position in array Exchange* a[mid] *and* a[last – 1]
pivotIndex = last – 1 pivotValue = a[pivotIndex]

- Adjusting the Partition Algorithm

Quick Sort Partitioning (Part 2)

```
// Determine two subarrays:
//          Smaller = a[first..endSmaller] and
//          Larger = a[endSmaller+1..last–1]
// such that entries in Smaller are <= pivotValue and entries in Larger are >= pivotValue.
//  Initially, these subarrays are empty.
indexFromLeft = first + 1 indexFromRight = last − 2
done = false
while (!done)
{
        // Starting at the beginning of the array, leave entries that are < pivotValue and
        // locate the first entry that is >= pivotValue. You will find one, since the last
        // entry is >= pivotValue.
        while (a[indexFromLeft] < pivotValue)
                indexFromLeft++
        // Starting at the end of the array, leave entries that are > pivotValue and
        // locate the first entry that is <= pivotValue. You will find one, since the first
        // entry is <= pivotValue.
        while (a[indexFromRight] > pivotValue)
                indexFromRight−−
```

- Adjusting the Partition Algorithm

Quick Sort Partitioning (Part 3)

```
// Assertion: a[indexFromLeft] >= pivotValue and
//                                a[indexFromRight] <= pivotValue
if (indexFromLeft < indexFromRight)
{
        Exchange a[indexFromLeft] and a[indexFromRight]
        indexFromLeft++
        indexFromRight−−
}
else
        done = true
}  // while (!done)
// Place pivotValue between the subarrays Smaller and Larger
Exchange a[pivotIndex] and a[indexFromLeft]
pivotIndex = indexFromLeft
// Assertion: Smaller = a[first..pivotIndex–1]
//            pivotValue = a[pivotIndex]
//            Larger = a[pivotIndex+1..last]
return pivotIndex
```

- Adjusting the Partition Algorithm

The Quick Sort Method

- Implementation of the quick sort.

```
public static <T extends Comparable<? super T>>
      void quickSort(T[] a, int first, int last)
{
  if (last - first + 1 < MIN_SIZE)
  {
    insertionSort(a, first, last);
  }
  else
  {
    // Create the partition: Smaller | Pivot | Larger
    int pivotIndex = partition(a, first, last);

    // Sort subarrays Smaller and Larger
    quickSort(a, first, pivotIndex - 1);
    quickSort(a, pivotIndex + 1, last);
  } // end if
} // end quickSort
```

Quick Sort in the Java Class Library

- Class **`Arrays`** in the package **`java.util`** defines versions of a static method sort

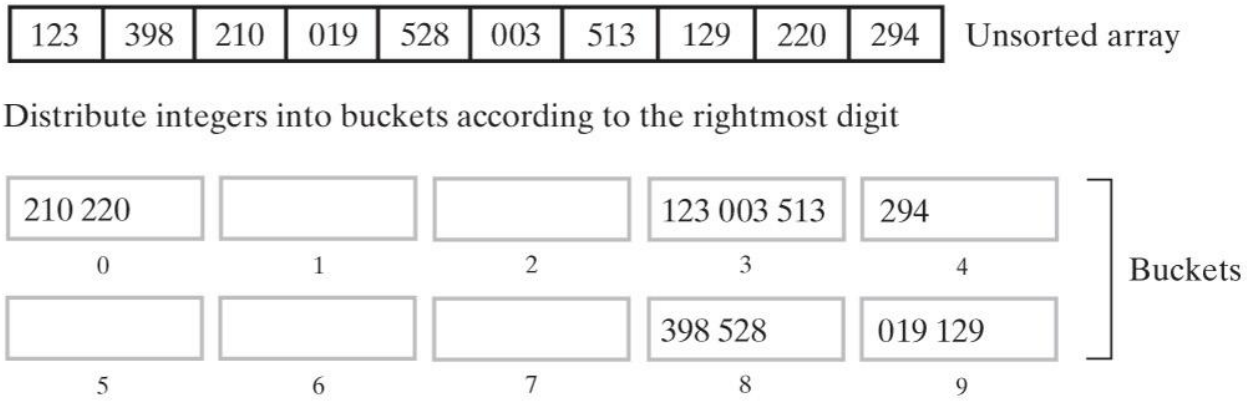
```
public static void sort(type[] a)

public static void sort(type[] a, int first, int after)
```

Radix Sort

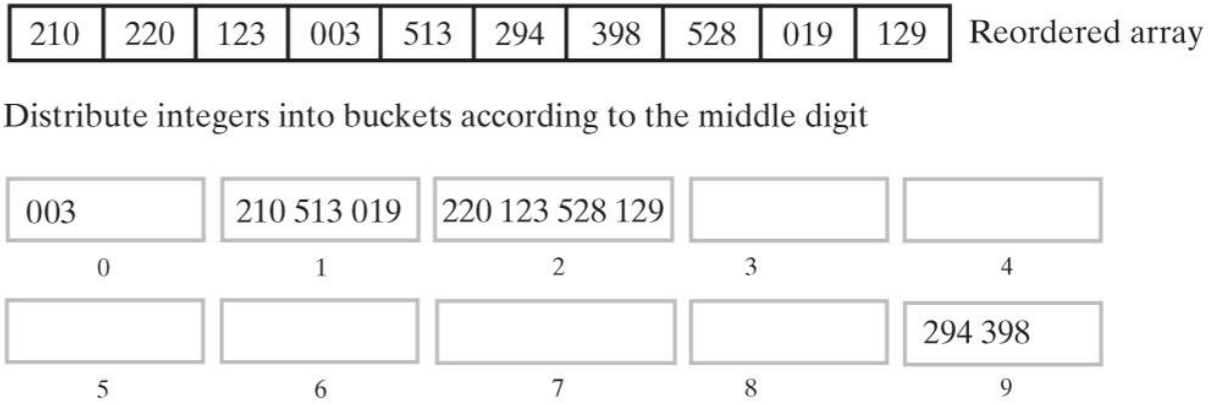
- Does not use comparison
- Treats array entries as if they were strings that have the same length.
  - Group integers according to their rightmost character (digit) into “buckets”
  - Repeat with next character (digit), etc.

(a) Distribution of the original array into buckets



© 2019 Pearson Education, Inc.

(b) Distribution of the reordered array into buckets



© 2019 Pearson Education, Inc.

Radix Sort (Part 2)

(c) Distribution of the reordered array into buckets

003	210	513	019	220	123	528	129	294	398

Reordered array

Distribute integers into buckets according to the leftmost digit

0	1	2	3	4
003 019	123 129	210 220 294	398	

5	6	7	8	9
513 528				

© 2019 Pearson Education, Inc.

(d) Sorting is complete

003	019	123	129	210	220	294	398	513	528

Sorted array

© 2019 Pearson Education, Inc.

- FIGURE 16-9 The steps of a radix sort

Algorithm Comparison

	$O(n)$	$O(n)$	$O(n)$
	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$
	$O(n \log n)$	$O(n \log n)$	$O(n^2)$
	$O(n)$	$O(n^{1.5})$	$O(n^{1.5})$
	$O(n)$	$O(n^2)$	$O(n^2)$
	$O(n^2)$	$O(n^2)$	$O(n^2)$

The time efficiency of various sorting algorithms, expressed in Big Oh notation

Comparing Function Growth Rates

- A comparison of growth-rate functions as n increases

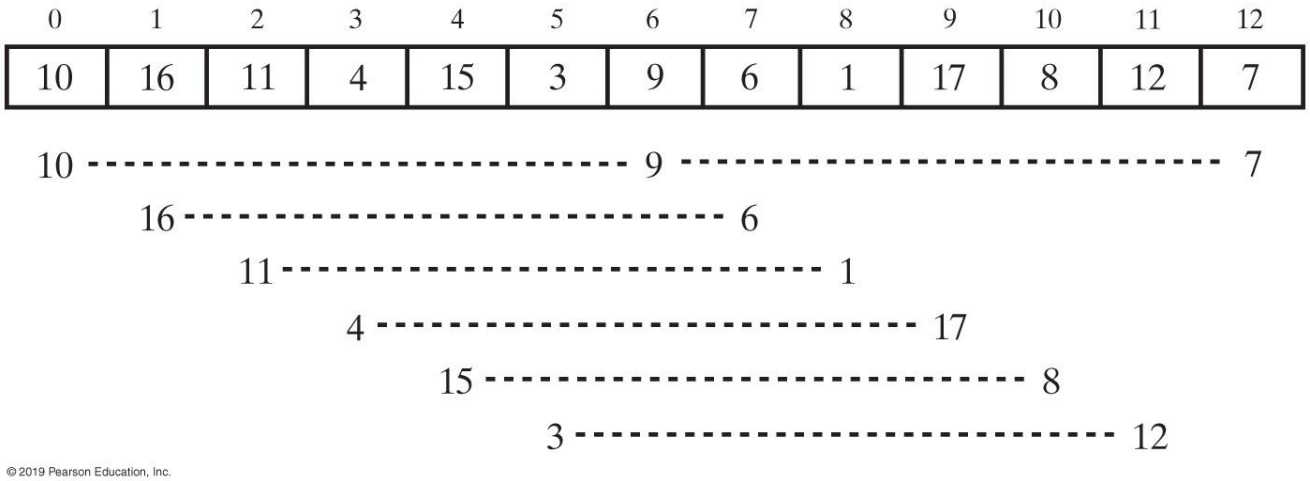
		10^2	10^3	10^4	10^5	10^6
n	10	100	1,000	10,000	100,000	1,000,000
$n \log n$	33	664	9,966	132,877	1,660,964	19,931,569
$n^{1.5}$	32	1,000	31,623	1,000,000	319,622,777	109
n^2	100	10,000	1,000,000	108	1,010	1,012

Shell Sort

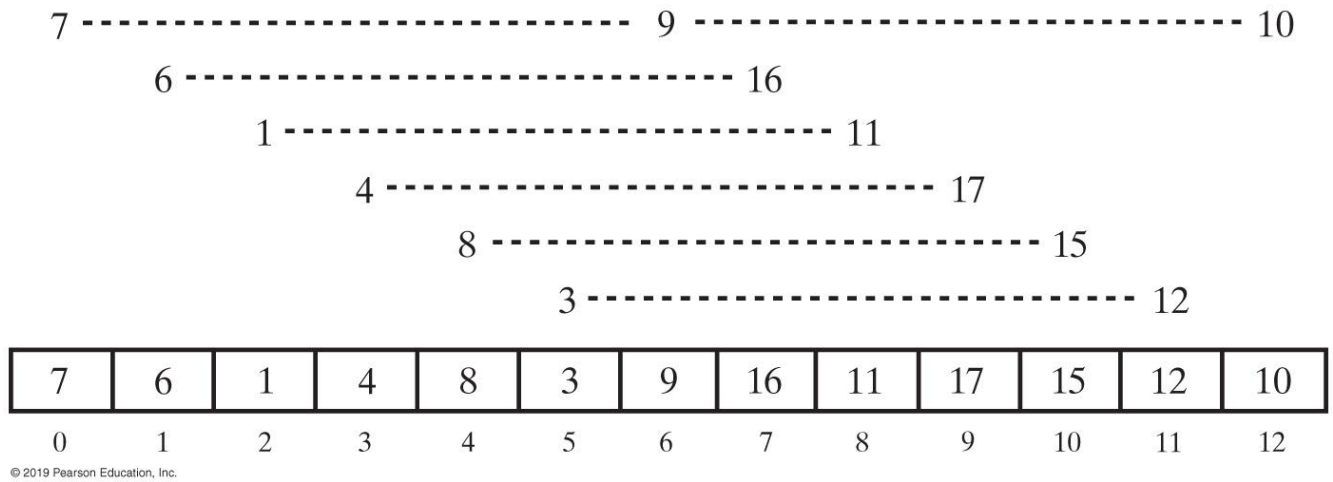
- Algorithms so far are simple
  - but inefficient for large arrays at $O(n^2)$
- The more sorted an array is, the less work `insertInOrder` must do
- Improved insertion sort developed by Donald Shell

Shell Sort

Before Ordering



After Ordering



- An array and the groups of entries whose indices are 6 apart before and after ordering groups

Shell Sort

0	1	2	3	4	5	6	7	8	9	10	11	12
7	6	1	4	8	3	9	16	11	17	15	12	10

- 7 - - - - - - - - - - - - - 4 - - - - - - - - - - - - - 9 - - - - - - - - - - - - - 17 - - - - - - - - - - - - - 10

Before Ordering

6 - - - - - - - - - - - - - 8 - - - - - - - - - - - - - 16 - - - - - - - - - - - - - 15

1 - - - - - - - - - - - - - 3 - - - - - - - - - - - - - 11 - - - - - - - - - - - - - 12

© 2019 Pearson Education, Inc.

4 - - - - - - - - - - - - - 7 - - - - - - - - - - - - - 9 - - - - - - - - - - - - - 10 - - - - - - - - - - - - - 17

After Ordering

6 - - - - - - - - - - - - - 8 - - - - - - - - - - - - - 15 - - - - - - - - - - - - - 16

1 - - - - - - - - - - - - - 3 - - - - - - - - - - - - - 11 - - - - - - - - - - - - - 12

4	6	1	7	8	3	9	15	11	10	16	12	17
0	1	2	3	4	5	6	7	8	9	10	11	12

© 2019 Pearson Education, Inc.

Comparing Algorithms

	$O(n^2)$	$O(n^2)$	$O(n^2)$
	$O(n)$	$O(n^2)$	$O(n^2)$
	$O(n)$	$O(n^{1.5})$	$O(n^{1.5})$

The time efficiencies of three sorting algorithms, expressed in Big Oh notation